Test-Driven Development (TDD)

Victor Eijkhout, Susan Lindsey

Fall 2026

last formatted: October 27, 2026

1. Dijkstra quote

> *Today a usual technique is to make a program and then to test it. But: program testing can be a very effective way to show the presence of bugs, but is hopelessly inadequate for showing their absence. (cue laughter)*

Still ...

Intro to testing

2. Types of testing

- *Unit tests* that test a small part of a program by itself;
- *System tests* test the correct behavior of the whole software system; and
- *Regression tests* establish that the behavior of a program has not changed by adding or changing aspects of it.

3. Unit testing

- Every part of a program should be testable
- $\Rightarrow$ good idea to have a function for each bit of functionality
- Positive tests: show that code works when it should
- Negative tests: show that the code fails when it should

4. Unit testing

- Every part of a program should be testable
- Do not write the tests after the program:
  write tests while you develop the program.
- Test-driven development:
  1. design functionality
  2. write test
  3. write code that makes the test work

5. Principles of TDD

Develop code and tests hand-in-hand:

- Both the whole code and its parts should always be testable.
- When extending the code, make only the smallest change that allows for testing.
- With every change, test before and after.
- Assure correctness before adding new features.

6. Unit testing frameworks

Testing is important, so there is much software to assist you.

Popular choice with C++ programmers: Catch2
`https://github.com/catchorg`

7. Compiling

```
1 icpc -o tdd tdd.cxx \
2     -I${TACC_CATCH2_INC} -L${TACC_CATCH2_LIB} \
3     -lCatch2Main -lCatch2
```

- Path to include files:

```
1   -I${TACC_CATCH2_INC}
2   # or:
3   $( pkg-config --cflags catch2 )
```

- Libraries:

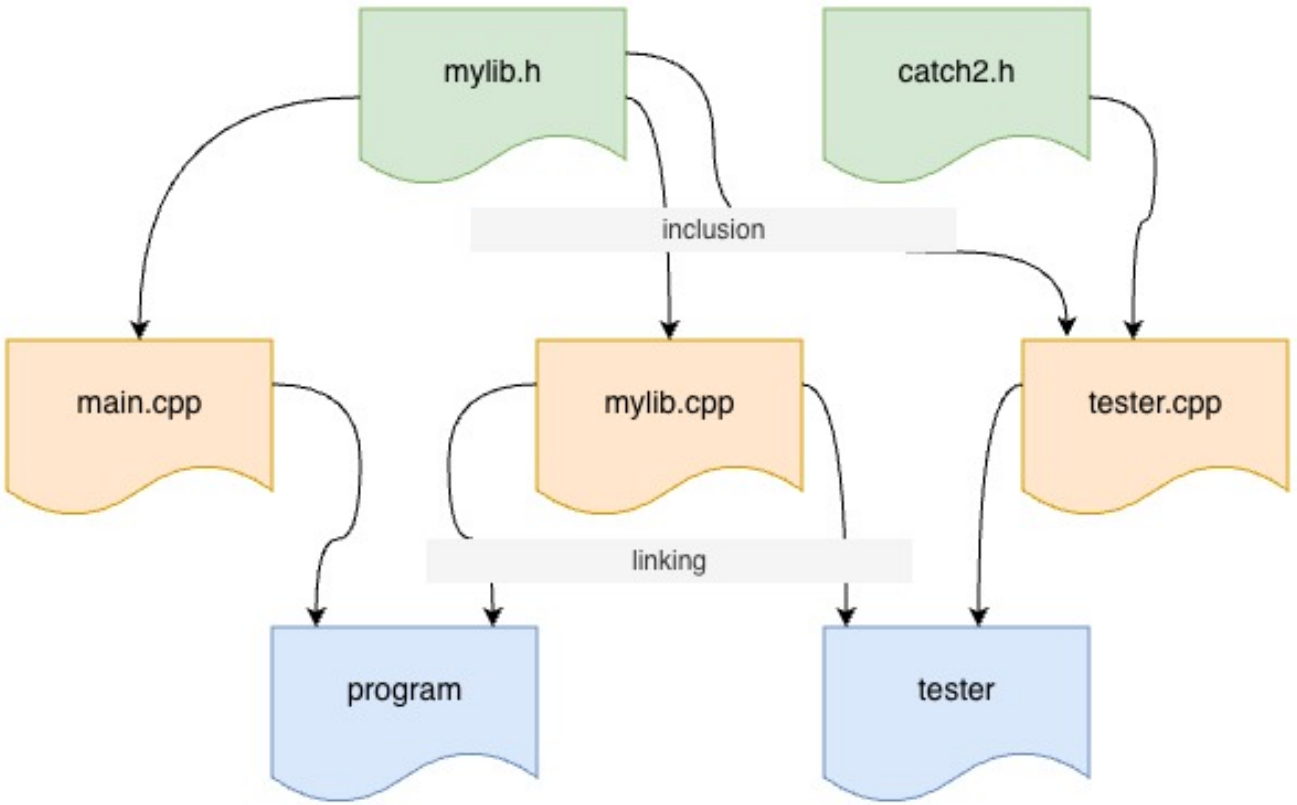
```
1   -L${TACC_CATCH2_LIB} -lCatch2Main -lCatch2
2   # or:
3   $( pkg-config --libs catch2-with-main )
```

8. CMake

```
1 find_package( PkgConfig REQUIRED )
2 pkg_check_modules( CATCH2 REQUIRED catch2-with-main )
3 target_include_directories(
4         myprogram PUBLIC
5         ${CATCH2_INCLUDE_DIRS}
6         )
7 target_link_directories(
8         myprogram PUBLIC
9         ${CATCH2_LIBRARY_DIRS}
10        )
11 target_link_libraries(
12        myprogram PUBLIC
13        ${CATCH2_LIBRARIES}
14        )
```

9. Realistic setup

- All program functionality in a 'library' file
- Main program really short
- Tester file with only tests.
- (Tester also needs the `catch2` stuff included)



Exercise 1: File structure

Make three files:

1. Include file with the functions.
2. Main program that uses the functions.
3. Tester main file, contents to be determined.

10. Minimal example

Function and tester:

```
1  // catch/positive1.cpp
2  #define CATCH_CONFIG_MAIN
3  #include "catch2/catch_all.hpp"
4
5  double f(int n) { return n*n+1; }
6
7  TEST_CASE( "test that f works once" ) {
8    int n{-3};
9    REQUIRE( f(n)>0 );
10 }
11 TEST_CASE( "test that f likely returns positive" ) {
12   for ( auto n : {-5,-3,-1,0,1,2,3,4,5} )
13     REQUIRE( f(n)>0 );
14 }
```

(accept the `define` and `include` as magic for now)

11. Correctness through 'require' clause

Tests go in *tester.cpp*:

```
1 TEST_CASE( "test that f always returns positive" ) {
2   for (int n=0; n<1000; n++)
3     REQUIRE( f(n)>0 );
4 }
```

- `TEST_CASE` acts like independent main program. can have multiple cases in a tester file
- `REQUIRE` is like `assert` but more sophisticated

Exercise 2

You are going to write a function *safe_sqrt*.
(Find a template in the `sessions/nov04` directory.)

Write one or two tests that confirm the correctness of your function.

12. Output for failing tests

What if your code is wrong?

Code:

```
1 // catch/positive3.cpp
2 // Wrong
3 double f(int n) { return n*n-2; }
4
5 TEST_CASE( "test that f returns
      positive" ) {
6   for ( auto n :
      {-5,-3,-1,0,1,2,3,4,5} ) {
7     INFO( "testing: " << n );
8     REQUIRE( f(n)>0 );
9   }
10 }
```

Output:

```
1 positive3.cpp:24:
      ↪FAILED:
2   REQUIRE( f(n)>0 )
3 with expansion:
4   -1.0 > 0
5 with message:
6   testing: -1
```

13. Tests

Boolean:

```
1 REQUIRE( some_test(some_input) );
2 REQUIRE( not some_test(other_input) );
```

Integer:

```
1 REQUIRE( integer_function(1)==3 );
2 REQUIRE( integer_function(1)!=0 );
```

Boolean expressions need to be parenthesized:

```
1 REQUIRE( ( x>0 and x<1 ) );
```

14. Diagnostic information for failing tests

Print out information at a failing test:

```
1 TEST_CASE( "test that f always returns positive" ) {
2   for (int n=0; n<1000; n++)
3     INFO( "iteration: " << n );
4     REQUIRE( f(n)>0 );
5 }
```

15. Generate inputs

```
1 // catch/positive2.cpp
2 TEST_CASE( "test that f returns positive" ) {
3   int n = GENERATE( 1,2,3,4,5 );
4   REQUIRE( f(n)>0 );
5 }
```

```
1 // catch/positive2.cpp
2 TEST_CASE( "test that f always returns positive" ) {
3   int n = GENERATE(take(100, random(-100, 100)));
4   REQUIRE( f(n)>0 );
5 }
```

```
1 // catch/positive2.cpp
2 TEST_CASE( "test that f always returns real positive" ) {
3   auto n = GENERATE(take(100, random(-100., 100.)));
4   cout << n << '\n';
5   REQUIRE( f(n)>0 );
6 }
```

Exercise 3

Extend exercise **??**y generating a range of inputs.

16. Numeric tests

Floating point numbers are hardly ever exact.
Do approximate tests:

```
1 REQUIRE( real_function(1.5)==Catch::Approx(3.0) );
2 REQUIRE( real_function(1)!=Catch::Approx(1.0) );
3 REQUIRE( zero_find()==Catch::Approx(0.).margin(1.e-8) );
```

Exercise 4: Safe sqrt 3

Extend your test by testing that your square root function delivers approximately the right result.

17. Diagnostic information for failing tests

Print out information at a failing test:

```
1 TEST_CASE( "test that f always returns positive" ) {
2   for (int n=0; n<1000; n++)
3     INFO( "iteration: " << n );
4     REQUIRE( f(n)>0 );
5 }
```

18. Test for exceptions

Suppose function $g(n)$

- succeeds for input $n > 0$
- fails for input $n \leq 0$:
  throws exception

```
1 TEST_CASE( "test that g only works for positive" ) {
2   for (int n=-100; n<+100; n++)
3     if (n<=0)
4       REQUIRE_THROWS( g(n) );
5     else
6       REQUIRE_NOTHROW( g(n) );
7 }
```

also *CHECK_THROWS* and *CHECK_NOTHROW*:
report error, but continue testing.

Exercise 5: Safe sqrt 4

Extend your function: a negative input should cause an exception.

Test for this.

19. Tests with code in common

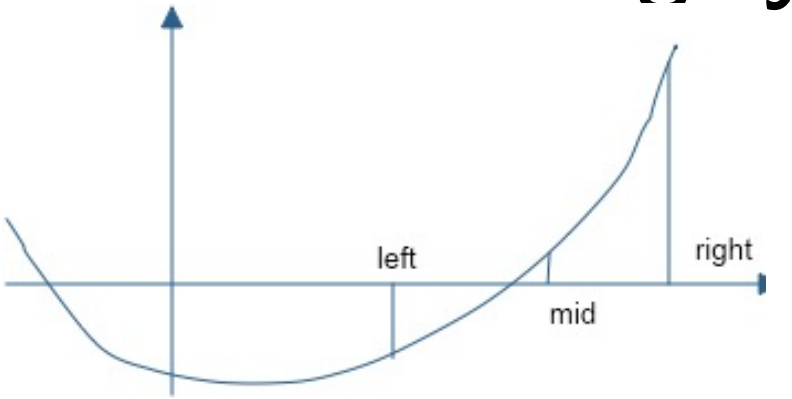
Use SECTION if tests have intro/outtro in common:

```
1  TEST_CASE( "commonalities" ) {
2    // common setup:
3    double x,y,z;
4    REQUIRE_NOTHROW( y = f(x) );
5    // two independent tests:
6    SECTION( "g function" ) {
7      REQUIRE_NOTHROW( z = g(y) );
8    }
9    SECTION( "h function" ) {
10     REQUIRE_NOTHROW( z = h(y) );
11   }
12   // common followup
13   REQUIRE( z>x );
14 }
```

(sometimes called setup/teardown)

TDD example: Bisection

20. Root finding by bisection



- Start with bounds where the function has opposite signs.

$$x_- < x_+, \qquad f(x_-) \cdot f(x_+) < 0,$$

- Find the mid point;
- Adjust either left or right bound.

21. Coefficient handling

$$f(x) = c_0 x^d + c_1 x^{d-1} \cdots + c_{d-1} x^1 + c_d$$

We implement this by storing the coefficients in a `vector<double>`.

Proper:

```
1 // root/testzeroarray.cpp
2 TEST_CASE( "coefficients represent polynomial" "[1]") {
3   vector<double> coefficients = { 1.5, 0., -3 };
4   REQUIRE( coefficients.size()>0 );
5   REQUIRE( coefficients.front()!=0. );
6 }
```

Exercise 6: One test for properness

For polynomial coefficients to give a well-defined polynomial, the zero-th coefficient needs to be non-zero:

```
1 // bisect/zeroclasstest.cpp
2 TEST_CASE( "proper test","[2]" ) {
3   vector<double> coefficients{3., 2.5, 2.1};
4   REQUIRE_NOTHROW( polynomial(coefficients) );
5
6   coefficients.at(0) = 0.;
7   REQUIRE_THROWS( polynomial(coefficients) );
8 }
```

Write a constructor that accepts the coefficients, and throws an exception if the above condition is violated.

22. Handy shortcut

Are you getting tired of typing `vector<double>`?
put

```
1 // root/findzerolib.hpp
2 using polynomial = vector<double>;
```

somewhere high in your file.

23. Test on polynomials evaluation

Next we need to evaluate polynomials.

Equality testing on floating point is dangerous:

use *Catch*::*Approx*(*sb*)

```
1 // root/testzeroclass.cpp
2 polynomial second( {2,0,1} );
3 // correct interpretation: 2x^2 + 1
4 REQUIRE( second.is_proper() );
5 REQUIRE( second.evaluate_at(2) == Catch::Approx(9) );
6 // wrong interpretation: 1x^2 + 2
7 REQUIRE( second.evaluate_at(2) != Catch::Approx(6) );
```

Exercise 7: Implementation

Write a function *evaluate_at* which computes

$$y \leftarrow f(x).$$

and confirm that it passes the above tests.

```
1 double evaluate_at( polynomial coefficients,double x);
```

For bonus points, look up Horner's rule and implement it.

24. Odd degree polynomials only

With odd degree you can always find bounds x_-, x_+.
For this exercise we reject even degree polynomials:

```
1 // root/findzeroarray.cpp
2 if ( not is_odd(coefficients) ) {
3   cout << "This program only works for odd-degree polynomials\n";
4   exit(1);
5 }
```

This test will be used later;
first we need to implement it.

Exercise 8: Odd degree testing

Implement the *is_odd* test.

Gain confidence by unit testing:

```
1 // root/testzeroarray.cpp
2 polynomial second{2,0,1}; // 2x^2 + 1
3 REQUIRE( not is_odd(second) );
4 polynomial third{3,2,0,1}; // 3x^3 + 2x^2 + 1
5 REQUIRE( is_odd(third) );
```

25. Finding initial bounds

We need a function *find_initial_bounds* which computes x_-, x_+ such that

$$f(x_-) < 0 < f(x_+) \quad \text{or} \quad f(x_+) < 0 < f(x_-)$$

(can you write that more compactly?)

```
1 void find_initial_bounds
2   ( polynomial coefficients,double &left,double &right);
```

Since we reject even degree polynomials,
throw an exception for those.

Exercise 9: Test for initial bounds

Unit test:

```
1 // root/testzeroarray.cpp
2 right = left+1;
3 polynomial second{2,0,1}; // 2x^2 + 1
4 REQUIRE_THROWS( find_initial_bounds(second,left,right) );
5 polynomial third{3,2,0,1}; // 3x^3 + 2x^2 + 1
6 REQUIRE_NOTHROW( find_initial_bounds(third,left,right) );
7 REQUIRE( left<right );
```

Can you add a unit test on the left/right values?

26. Move the bounds closer

Root finding iteratively moves the initial bounds closer together:

```
1 move_bounds_closer(coefficients,left,right);
```

- on input, $left < right$, and
- on output the same must hold.

Design a test for this function;
implement this function.

27. Putting it all together

Ultimately we need a top level function

```
1 double find_zero( polynomial coefficients,double prec );
```

- reject even degree polynomials
- set initial bounds
- move bounds closer until close enough: $|f(y)| < \texttt{prec}$.

Exercise 10: Put it all together

Make this call work:

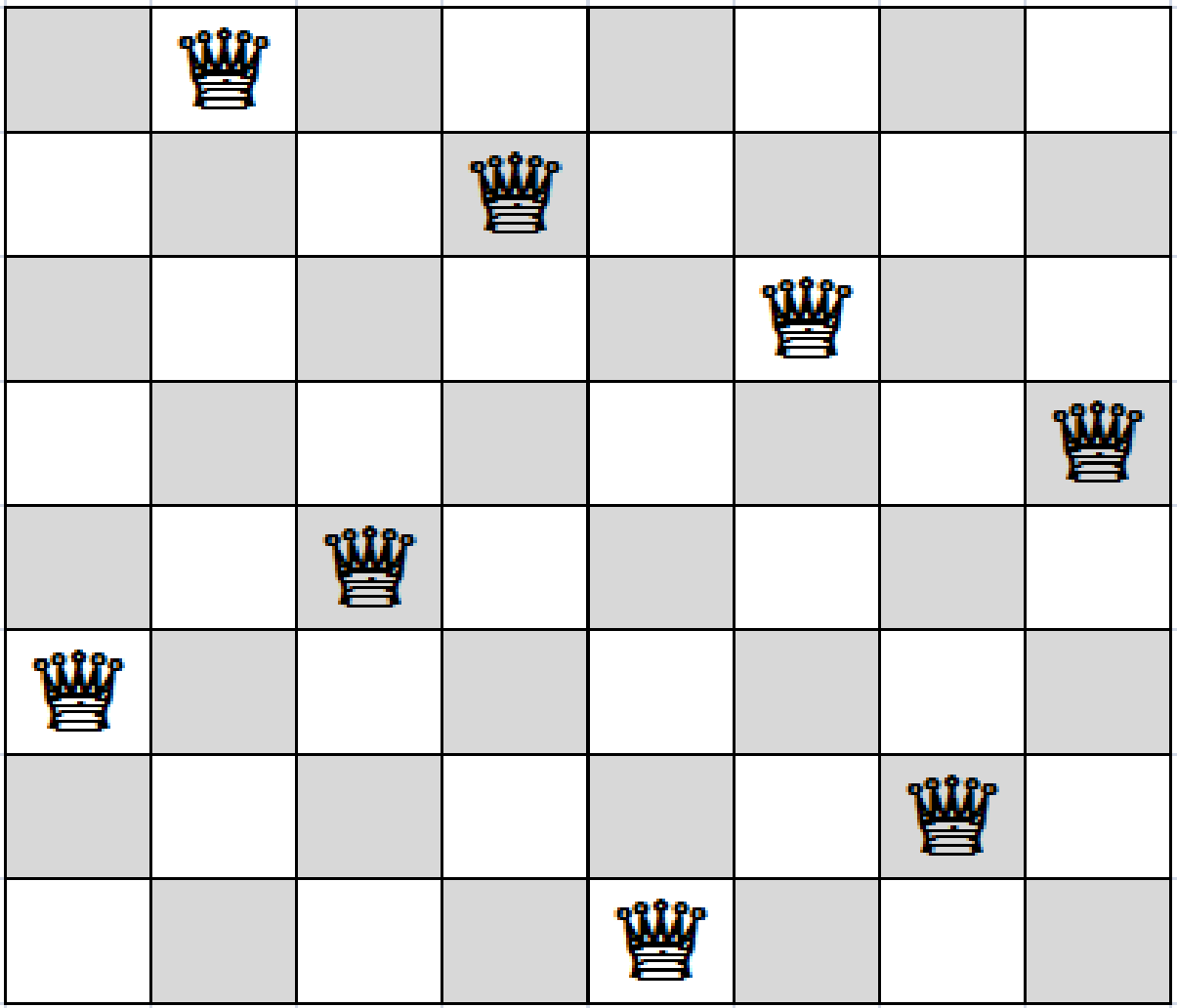
```
1 // root/findzeroarray.cpp
2 auto zero = find_zero( coefficients, 1.e-8 );
3 cout << "Found root " << zero
4      << " with value " << evaluate_at(coefficients,zero) << '\n';
```

Design unit tests, including on the precision attained, and make sure your code passes them.

Eight queens problem by TDD (using objects)

28. Problem statement

Can you place eight queens on a chess board so that no pair threatens each other?

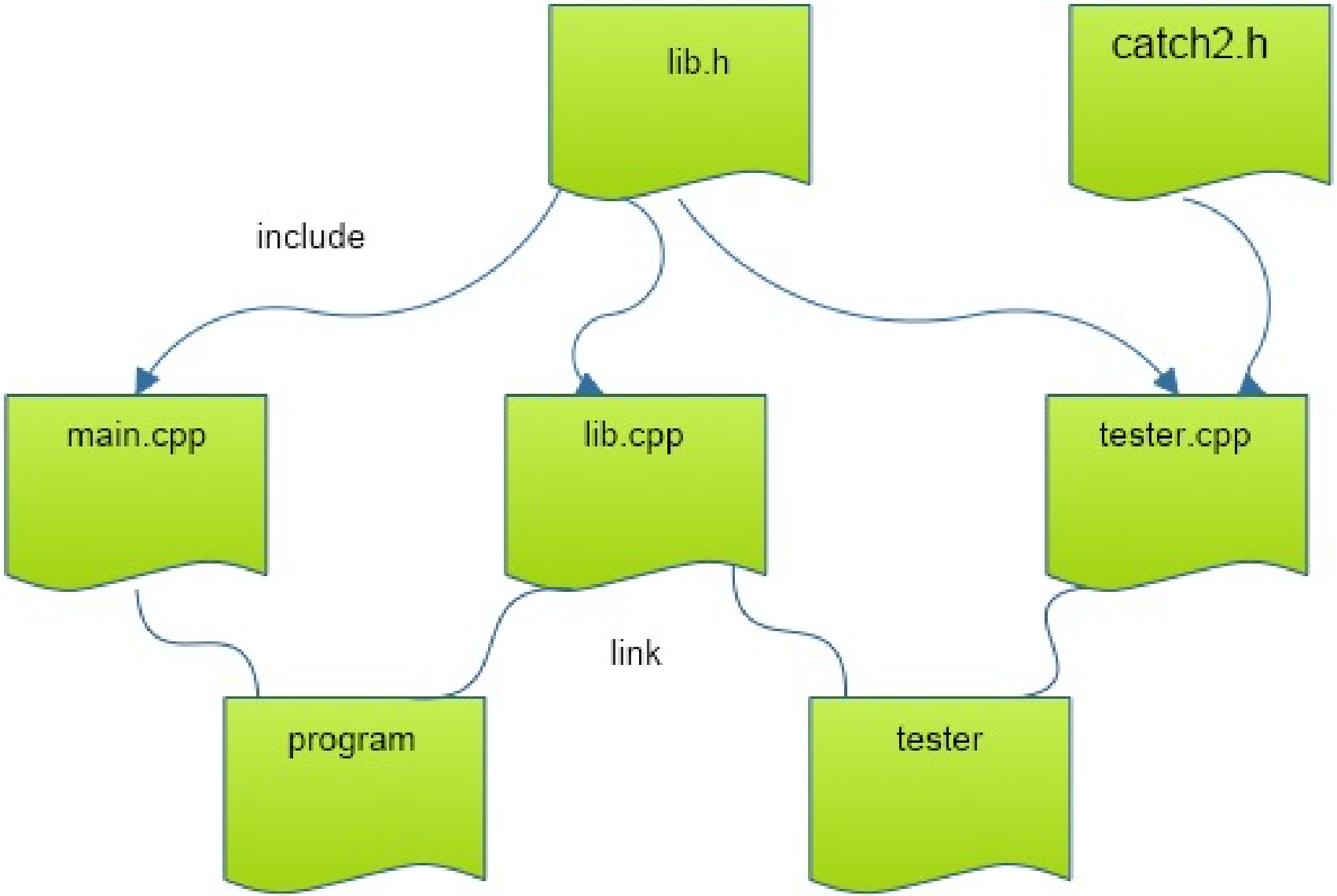


29. Sort of test-driven development

You will solve the ‘eight queens’ problem by

- designing tests for the functionality
- then implementing it

30. File structure



31. Basic object design

Object constructor of an empty board:

```
1 // queens/queenlib.hpp
2 ChessBoard(int n);
```

Test how far we are:

```
1 // queens/queenlib.hpp
2 int next_row_to_be_filled()
```

First test:

```
1 // queens/queentest.cpp
2 TEST_CASE( "empty board","[1]" ) {
3   constexpr int n=10;
4   ChessBoard empty(n);
5   REQUIRE( empty.next_row_to_be_filled()==0 );
6 }
```

Exercise 11: Board object

Start writing the *board* class, and make it pass the above test.

Exercise 12: Board method

Write a method for placing a queen on the next row,

```
1 // queens/queenlib.hpp
2 void place_next_queen_at_column(int i);
```

and make it pass this test (put this in a `TEST_CASE`):

```
1 // queens/queentest.cpp
2 INFO( "Illegal placement throws" );
3 REQUIRE_THROWS( empty.place_next_queen_at_column(-1) );
4 REQUIRE_THROWS( empty.place_next_queen_at_column(n) );
5 INFO( "Correct placement succeeds" );
6 REQUIRE_NOTHROW( empty.place_next_queen_at_column(0) );
7 REQUIRE( empty.next_row_to_be_filled()==1 );
```

Exercise 13: Test for collisions

Write a method that tests if a board is collision-free:

```
1 // queens/queenlib.hpp
2 bool feasible()
```

This test has to work for simple cases to begin with. You can add these lines to the above tests:

```
1 // queens/queentest.cpp
2 ChessBoard empty(n);
3 REQUIRE( empty.feasible() );
```

```
1 // queens/queentest.cpp
2 ChessBoard one = empty;
3 one.place_next_queen_at_column(0);
4 REQUIRE( one.next_row_to_be_filled()==1 );
5 REQUIRE( one.feasible() );
```

```
1 // queens/queentest.cpp
2 ChessBoard collide = one;
3 // place a queen in a 'colliding' location
4 collide.place_next_queen_at_column(0);
5 // and test that this is not feasible
6 REQUIRE( not collide.feasible() );
```

Exercise 14: Test full solutions

Make a second constructor to 'create' solutions:

```
1 // queens/queenlib.hpp
2 ChessBoard( int n,std::vector<int> cols );
3 ChessBoard( std::vector<int> cols );
```

Now we test small solutions:

```
1 // queens/queentest.cpp
2 ChessBoard five( {0,3,1,4,2} );
3 REQUIRE( five.feasible() );
```

Exercise 15: No more delay: the hard stuff!

Write a function that takes a partial board, and places the next queen:

```
1 // queens/queenlib.hpp
2 std::optional<ChessBoard> place_queens()
```

Test that the last step works:

```
1 // queens/queentest.cpp
2 ChessBoard almost( 4, {1,3,0} );
3 auto solution = almost.place_queens();
4 REQUIRE( solution.has_value() );
5 REQUIRE( solution->filled() );
```

Alternative to using `optional`:

```
1 bool place_queen( const board& current, board &next );
2 // true if possible, false is not
```

Exercise 16: Test that you can find solutions

Test that there are no 3×3 solutions:

```
1 // queens/queentest.cpp
2 TEST_CASE( "no 3x3 solutions","[9]" ) {
3   ChessBoard three(3);
4   auto solution = three.place_queens();
5   REQUIRE( not solution.has_value() );
6 }
```

but 4×4 solutions do exist:

```
1 // queens/queentest.cpp
2 TEST_CASE( "there are 4x4 solutions","[10]" ) {
3   ChessBoard four(4);
4   auto solution = four.place_queens();
5   REQUIRE( solution.has_value() );
6 }
```